

觸發器（Triggers）與別名（Aliases）使用指南

適用版本：mudclient-rs (Rust + Lua 架構)

最後更新：2026-03-03

目錄

- 1. [概念說明](#)
- 2. [別名 \(Aliases\)](#)
 - [基本語法](#)
 - [參數佔位符](#)
 - [腳本模式](#)
 - [透過 GUI 管理](#)
 - [透過 Profile JSON 設定](#)
- 3. [觸發器 \(Triggers\)](#)
 - [匹配模式](#)
 - [動作類型](#)
 - [正則捕獲群組](#)
 - [腳本模式](#)
 - [透過 GUI 管理](#)
 - [透過 Profile JSON 設定](#)
- 4. [全域設定 vs Profile 設定](#)
- 5. [Lua 腳本 API](#)
- 6. [實用範例](#)
- 7. [常見問題](#)

概念說明

功能	方向	說明
別名 (Alias)	輸出（你打字）	將你鍵入的短指令展開為完整指令再送出
觸發器 (Trigger)	輸入（伺服器傳來）	偵測伺服器訊息，自動執行指定動作

別名（Aliases）

基本語法

別名由三個部分組成：

欄位	說明	範例
名稱 (name)	識別用，不影響比對	kk
模式 (pattern)	你輸入的文字樣板	kk

替換 (replacement)	展開後實際送出的指令	kill kobold
------------------	------------	-------------

注意：別名採用完整行比對（整行必須符合模式），輸入 `kk` 才觸發，輸入 `kka` 不會觸發。

範例：簡單縮寫

名稱：kk
模式：kk
替換：kill kobold

輸入 `kk` → 自動送出 `kill kobold`

參數佔位符

在模式中加入 `$1`、`$2` 或 `$*` 捕獲參數，在替換中引用。

佔位符	說明
<code>\$1</code>	第一個空格分隔的參數
<code>\$2</code>	第二個參數
<code>\$*</code>	匹配所有剩餘參數

選用參數：若模式為 `cfr $1`（`$1` 前有空格），則不帶參數時 `cfr` 也能匹配，`$1` 展開為空字串。

必填參數：若模式為 `go$1`（`$1` 前無空格），則必須帶參數才能匹配。

範例：單一參數

名稱：go
模式：go \$1
替換：walk \$1;look

輸入 `go north` → 送出 `walk north;look`

範例：多個參數

名稱：cast
模式：c \$1 \$2
替換：cast \$1 at \$2

輸入 `c fireball goblin` → 送出 `cast fireball at goblin`

範例：全部參數（`$*`）

名稱：say
模式：s \$*
替換：say \$*

輸入 `s hello world` → 送出 `say hello world`

範例：選用參數

```
名稱: cfr
模式: cfr $1
替換: c 'full ref' $1
```

- 輸入 `cfr target` → 送出 `c 'full ref' target`
- 輸入 `cfr` → 送出 `c 'full ref'`

別名腳本模式

將「替換」欄位填入 Lua 程式碼，並勾選「腳本模式」，可執行任意邏輯：

```
-- 別名腳本範例：輸入 "hp" 顯示目前 HP 並發送 score
mud.send("score")
mud.echo("[Script] 已發送 score 指令")
```

腳本模式中可使用完整的 `mud.*` API，詳見 [Lua 腳本 API](#)。

透過 GUI 管理別名

1. 開啟設定面板 → **別名 (Aliases)** 分頁
2. 點選「新增」按鈕
3. 填入「名稱」、「模式」、「替換」
4. 若使用 Lua 程式碼，勾選「腳本模式」
5. 可設定「分類」方便整理（例如 `戰鬥`、`移動`）
6. 勾選「啟用」後儲存即生效

優先順序：模式越長的別名優先匹配，避免短模式意外截斷長模式。

透過 Profile JSON 設定別名

Profile 檔案位於 `~/.config/mudclient/profiles/<名稱>.json`：

```
{
  "aliases": [
    {
      "name": "kk",
      "pattern": "kk",
      "replacement": "kill kobold",
      "enabled": true,
      "is_script": false,
      "category": "戰鬥"
    },
    {
      "name": "go",
```

```
    "pattern": "go $1",
    "replacement": "walk $1;look",
    "enabled": true,
    "is_script": false
  },
  {
    "name": "hp_script",
    "pattern": "hp",
    "replacement": "mud.send('score'); mud.echo('查詢中...')",
    "enabled": true,
    "is_script": true
  }
]
```

觸發器（Triggers）

匹配模式

觸發器在偵測伺服器訊息時，支援四種比對方式：

模式	說明	適用場景
Contains	訊息中包含指定字串	關鍵字偵測（最常用）
StartsWith	訊息開頭符合指定字串	偵測特定格式的系統訊息
EndsWith	訊息結尾符合指定字串	偵測特定結尾標記
Regex	正則表達式完整比對	需要捕獲數字或變動內容時

自動偵測：在 GUI 或 JSON 設定時，若 pattern 包含 `(.+)`、`\d`、`[`、`^`、`$`、`|`、`?` 等字元，系統會自動切換為 Regex 模式。

ANSI 顏色處理：觸發器比對前會自動剝除 ANSI 顏色碼，無需手動處理。

動作類型

觸發器目前支援以下動作：

動作	說明	使用場景
SendCommand	發送指令到 MUD 伺服器	自動回應、自動施法
ExecuteScript	執行 Lua 腳本	複雜邏輯、條件判斷
Highlight	高亮顯示（RGB 前景色）	標記危險訊息
Gag	隱藏該行訊息	過濾雜訊、廣告

PlaySound	播放音效（路徑）	緊急提醒
RouteToWindow	路由到子視窗	分頻道顯示

多動作：一個觸發器可以同時設定多個動作（例如高亮 + 發送指令）。

正則捕獲群組

使用 Regex 模式時，括號（...）內捕獲的內容，可在：

- **SendCommand** 動作中用 \$1 、 \$2 引用
- **Lua** 腳本中用 captures[1] 、 captures[2] 引用

範例：捕獲金幣數量

模式（Regex）：你獲得了\s*(\d+)\s*金幣

動作（SendCommand）：count gold --（捕獲的 \$1 = 金幣數量）

若訊息為「你獲得了 100 金幣！」， captures[1] = "100"

多捕獲群組：

模式（Regex）：你的生命值還剩下 (\d+)/(\d+)

- captures[1] = 當前 HP
- captures[2] = 最大 HP

觸發器腳本模式

將「動作」欄位填入 Lua 程式碼，並勾選「腳本模式」：

```
-- 觸發器腳本範例：HP 低於 30% 時自動喝藥
local current_hp = tonumber(captures[1])
local max_hp = tonumber(captures[2])

if current_hp < (max_hp * 0.3) then
    mud.echo("⚠ 生命危急！")
    mud.send("drink health_potion")
end
```

在腳本中可使用以下特殊變數：

變數	說明
captures[1].....	正則捕獲群組
message	觸發的原始訊息行
variables["key"]	持久化全域變數

透過 GUI 管理觸發器

1. 開啟設定面板 → 觸發器 (Triggers) 分頁
 2. 點選「新增」按鈕
 3. 填入「名稱」和「模式 (Pattern)」
 4. 填入「動作 (Action)」欄位：
 - 一般模式：填入要發送的指令
 - 腳本模式：勾選「腳本模式」，填入 Lua 程式碼
 5. 可設定「分類」(例如 治療、過濾)
 6. 透過 `mud.enable_trigger("名稱", true/false)` 動態啟用/停用
-

透過 Profile JSON 設定觸發器

```
{
  "triggers": [
    {
      "name": "auto_heal",
      "pattern": "你的生命值還剩下 (\\d+)/ (\\d+)",
      "action": "local hp = tonumber(captures[1])\nlocal max =
tonumber(captures[2])\nif hp < max * 0.3 then mud.send('drink potion')
end",
      "enabled": true,
      "is_script": true,
      "category": "治療"
    },
    {
      "name": "gag_ads",
      "pattern": "廣告",
      "action": "mud.gag_message()",
      "enabled": true,
      "is_script": true,
      "category": "過濾"
    },
    {
      "name": "gold_notify",
      "pattern": "你獲得了\\s*(\\d+)\\s*金幣",
      "action": "mud.echo('獲得 ' .. captures[1] .. ' 金幣!')",
      "enabled": true,
      "is_script": true,
      "category": "通知"
    }
  ]
}
```

JSON 中的正則：反斜線需要雙重跳脫，例如 `\d` 寫成 `\\d`。

全域設定 vs Profile 設定

範圍	設定檔位置	說明
Profile 專屬	~/.config/mudclient/profiles/<名稱>.json	只在此帳號連線時生效
全域（所有帳號）	~/.config/mudclient/global_config.json	所有 Profile 連線時都生效

優先順序：Profile 專屬設定 > 全域設定（同名別名/觸發器時，Profile 優先）

全域設定 JSON 格式：

```
{
  "global_aliases": [
    {
      "name": "n",
      "pattern": "n",
      "replacement": "north",
      "enabled": true,
      "is_script": false
    }
  ],
  "global_triggers": [
    {
      "name": "global_warn",
      "pattern": "你受傷了",
      "action": "mud.echo('⚠ 受傷警告')",
      "enabled": true,
      "is_script": true
    }
  ]
}
```

Lua 腳本 API

在別名或觸發器的腳本模式中，可使用以下 API：

函數	說明
<code>mud.send("指令")</code>	發送指令到 MUD 伺服器
<code>mud.echo("文字")</code>	在視窗顯示訊息（不發送到伺服器）
<code>mud.log("文字")</code>	寫入系統日誌

mud.gag_message()	隱藏當前行（觸發器中使用）
mud.window("視窗名", "文字")	輸出到指定子視窗
mud.timer(秒數, "Lua 程式碼")	延遲執行
mud.enable_trigger("名稱", bool)	動態啟用/停用觸發器

全域變數表：

```
variables["target"] = "orc"      -- 寫入
local t = variables["target"]    -- 讀取
```

實用範例

範例 1：方向縮寫別名

```
{ "name": "n", "pattern": "n", "replacement": "north" },
{ "name": "s", "pattern": "s", "replacement": "south" },
{ "name": "e", "pattern": "e", "replacement": "east" },
{ "name": "w", "pattern": "w", "replacement": "west" }
```

範例 2：帶參數的施法別名

名稱：cf
 模式：cf \$1 \$2
 替換：cast 'fireball' \$1 \$2

輸入 cf goblin leader → 送出 cast 'fireball' goblin leader

範例 3：自動治療觸發器（Lua 腳本）

名稱：auto_heal
 模式（Regex）：你的生命值還剩下 (\d+)/(\d+)
 動作（Lua 腳本）：

```
local hp = tonumber(captures[1])
local max_hp = tonumber(captures[2])

if hp < max_hp * 0.3 then
  mud.echo("⚠ 生命危急！自動治療...")
  mud.send("cast 'heal' self")
  -- 2 秒後再檢查
```



```
mud.timer(2.0, "mud.send('score')")  
end
```

範例 4：過濾廣告訊息（Gag）

名稱：gag_ads
模式：廣告
動作（Lua 腳本）：mud.gag_message()

所有包含「廣告」的行將被自動隱藏。

範例 5：記錄目標並攻擊

名稱：kt
模式：kt \$1
動作（Lua 腳本）：

```
variables["target"] = "$1"  
mud.send("kill $1")  
mud.echo("[Script] 目標設定為：$1")
```

注意：別名腳本模式中，\$1 在執行前已被替換為實際輸入值。

範例 6：動態啟用/停用觸發器

```
-- 進入戰鬥時啟用自動治療觸發器  
mud.enable_trigger("auto_heal", true)  
  
-- 離開戰鬥時停用  
mud.enable_trigger("auto_heal", false)
```

範例 7：訊息分流到子視窗

名稱：route_chat
模式：聊天頻道
動作（Lua 腳本）：mud.window("chat", message)

常見問題

Q: 別名沒有觸發？

- 確認模式與輸入**完全匹配**（別名是全行比對）
- 若輸入 kk 卻設定模式 kk （後面有空格），會無法匹配
- 確認別名已啟用（enabled: true）

Q: 觸發器沒有觸發？

- 確認觸發器的模式與伺服器訊息的純文字（去除顏色碼後）符合
- Regex 模式請確認正則表達式語法正確，可先在 regex101.com 測試
- JSON 中的 `\d` 需寫成 `\\d`

Q: 正則捕獲群組索引是從 0 還是 1 開始？

從 1 開始。`captures[1]` 是第一個括號捕獲的內容。

Q: 多個觸發器都匹配同一行訊息，哪個先執行？

依加入順序依序執行，所有匹配的觸發器都會執行（不會只執行第一個）。

Q: 如何在腳本中讀取 `$var` 變數？

```
-- 使用 #var target goblin 設定後
local t = variables["target"]  -- 讀取
mud.send("kill " .. t)
```

Q: 別名模式支援萬用字元嗎？

支援 `*` 作為萬用字元（匹配任意字元）：

```
模式: look*
替換: look
```

此模式會匹配 `look`、`looknorth` 等以 `look` 開頭的輸入。

參見

- [API 指令說明](#) — 完整 Lua API 與 `#` 指令參考
- [腳本撰寫指南](#) — 進階 Lua 腳本架構（Hook、Timer、模組化）
- [腳本使用手冊](#) — 內建腳本說明（AutoCast、Practice 等）